

FTM SEO Automation

WordPress Metadata and Tag Automation - Comprehensive Technical Overview

A client-readable and engineer-ready reference for AI-assisted SEO generation, WordPress and AIOSEO updates, page and post matching, asynchronous jobs, reporting, operational health, verification, security boundaries, and current system limitations.

PHP | Python Flask | WordPress REST API | AIOSEO | Claude | Async Jobs | CSV Reporting | n8n History

Public technical edition - Evidence reconciled through 24 July 2026

1. Reading Guide and Publication Boundary

This document serves business and technical readers. Client stakeholders can focus on the executive overview, operator workflow, current status, safeguards, outcomes, and limitations. Engineers can continue through the request lifecycle, WordPress contracts, AI output controls, asynchronous processing, failure behavior, deployment verification, and representative traces. The content is intentionally safe for broad professional review. It describes system behavior without exposing passwords, application credentials, API secrets, access tokens, private host details, private resource identifiers, internal file paths, private test sites, account usernames, or administrative login instructions. Authorized maintainers obtain those values from approved deployment configuration and secret storage.

Truth conventions used throughout:

- Current implementation - behavior supported by the live PHP and Python sources and the newest project evidence.
- Historical workflow - earlier n8n-based behavior retained as engineering context, not described as the only current production path.
- Historical validation - a dated result proving a specific build and test scope.
- Known limitation - a verified boundary or hardening opportunity stated directly.

2. Executive Overview

Fine Touch Marketing's SEO automation reduces repetitive WordPress optimization work. An operator supplies an authorized WordPress site, selects pages or posts, and submits one or more existing titles with optional location context. The system matches those titles to WordPress content, generates structured SEO copy, validates and repairs the output, saves metadata through AIOSEO, assigns relevant WordPress tags when supported, and returns a per-item result summary.

The current operational path uses a protected PHP web interface in front of a separately supervised Python API and job runner. The browser receives a job identifier quickly and polls for status while the worker performs the longer WordPress and AI operations. This prevents a long SEO run from depending on one browser request remaining open.

The project also retains a substantial n8n workflow history. That workflow established title matching, structured AI generation, AIOSEO updates, tag creation, and result aggregation. The current technical reference distinguishes that historical automation path from the newer live PHP and Python runtime.

Current high-level characteristics:

- Protected operator interface with health, run, and job-status actions.
- Asynchronous Python jobs for page or post optimization.
- WordPress REST discovery, matching, metadata updates, and optional content creation.
- AIOSEO title, description, focus keyphrase, keyword, social metadata, and analysis support.
- Industry-locked and location-aware AI generation with deterministic repair rules.
- CSV and user-interface summaries for completed, failed, and skipped items.
- Health-based supervision for the backend API rather than process-name checks alone.

3. Product Purpose, Users, and Outcomes

The product helps an authorized marketing operator optimize many existing WordPress pages or posts without opening and editing each item manually. It is designed for repeatable metadata work across client sites while preserving site-specific industry, location, and service context.

Primary users:

- Marketing operators who submit approved page or post targets.
- Client stakeholders who review SEO wording and outcome reports.
- WordPress administrators who provision least-privilege access and required plugins.
- Engineers who maintain generation rules, API behavior, deployment health, and regression evidence.

Explicit non-goals:

- No guarantee of search ranking, traffic, lead volume, or indexing time.
- No permission to optimize an unapproved site or content item.
- No publication of private credentials in documentation, logs, reports, or screenshots.
- No replacement for client review of claims, location accuracy, branding, or regulated wording.
- No assumption that metadata saved through an API is immediately reflected in every cache or search engine.

4. Current Verified State

The current production project is in live maintenance. The protected PHP interface proxies authenticated operator actions to a local Python API. The API starts background jobs, tracks their state in memory, exposes safe status summaries, and invokes the SEO worker. The worker

connects to WordPress, validates AIOSEO availability, matches content, generates and repairs copy, saves results, assigns tags on a best-effort basis, and produces a report.

Verified implementation characteristics:

- The public interface and backend health are checked separately. A page loading successfully does not prove the worker API is available.
- A health-based watchdog can start the API when the service is actually unavailable.
- Local and deployed application sources were compared during the latest recovery evidence.
- Page and post flows have both been exercised in the workflow history.
- Title normalization covers common HTML entities, punctuation variation, and dash differences before matching.
- The AI output path constrains industry, location, title length, description length, focus keyword shape, and keyword count.
- AIOSEO save success is separated from optional WordPress tag behavior.

Historical workflow evidence includes a three-title run with all three targets matched and updated, a forty-tag concurrency repair, entity-clean output, and successful page and post verification. Those dated results do not replace current smoke checks after deployment or configuration changes.

5. Current Architecture

Operator browser -> protected PHP interface -> local Python API -> background SEO worker
SEO worker -> WordPress discovery -> content match -> AI generation -> deterministic repair
-> AIOSEO save

SEO worker -> WordPress tag resolution -> item update -> sitemap notification -> CSV and status summary

The PHP layer owns the operator session, the user interface, and the same-origin proxy boundary. It forwards approved health, run, and status actions to the locally reachable Python service. The browser does not need direct access to the worker port.

The Flask API owns job creation, background execution, lifecycle state, time limits, safe result parsing, and health reporting. The automation module owns WordPress connectivity, content selection, generation and repair logic, AIOSEO persistence, tag handling, and report creation. The earlier n8n workflow remains relevant for historical tests and reusable design decisions, but operational documentation must not confuse its cloud workflow identity with the active server-side job path.

6. Operator Workflow

1. Open the protected SEO tool and confirm that backend health is available.
2. Enter the authorized WordPress site address and least-privilege application credential.
3. Choose whether the targets are pages or posts.
4. Enter one existing title per line, optionally pairing a target with approved location context.
5. Select or confirm the correct business industry.
6. Submit the job and retain the returned job identifier while the interface polls status.

7. Review each completed, failed, skipped, or unmatched result.
 8. Spot-check saved metadata in WordPress and review wording before broader use.
- The interface can accept multiple targets in one job, but each target is resolved and reported independently. One unmatched title should not be presented as a successful update merely because other targets completed.

7. End-to-End Job Lifecycle

```
Validate input -> create job ID -> start background worker -> preflight WordPress -> resolve targets -> process each item -> report
```

The API validates required input, writes the submitted target list to a temporary job-specific source, constructs the worker request, records an in-memory job object, and starts background execution. The initial response returns before the full SEO operation finishes.

The browser polls by job identifier. While work is active, the interface shows progress rather than holding the original request open. When the worker reaches a terminal state, the API parses its output into page-level results and a summary suitable for the interface.

Terminal states include completion, failure, execution error, and timeout. A finished job is not automatically a successful SEO update; the per-item outcome and final counts remain authoritative.

8. WordPress Preflight and Access Boundary

Before changing content, the worker verifies that the site responds to authenticated WordPress REST requests and that the required SEO API surface is available. It confirms the selected content type and obtains the content collection needed for matching.

Access principles:

- Use a dedicated WordPress application credential for API work rather than a reusable primary login secret.
- Grant only the permissions required to read targets and update approved SEO fields.
- Treat an optional interactive administrator credential as a separate, higher-risk capability.
- Never include credential values in client exports, result summaries, command output, or diagnostic logs.
- Rotate credentials after suspected disclosure and remove access when a site leaves scope.

Credential-handling hardening remains important because worker processes, job launch mechanisms, and logs can accidentally expand exposure if arguments or exception details are recorded. Secret transport and retention should be reviewed independently of functional correctness.

9. Content Discovery and Title Matching

The system fetches WordPress pages or posts and builds a normalized lookup from their rendered titles. Both the stored title and the operator input pass through the same normalization contract before comparison.

Normalization covers:

- HTML entity decoding and removal of markup artifacts.
- Whitespace trimming and repeated-space collapse.
- Case-insensitive comparison.
- Equivalent dash and punctuation handling.
- Stable interpretation of one-title-per-line batch input.

A target that cannot be matched receives an explicit not-found result. The automation should not guess a loosely similar page ID because saving correct metadata to the wrong page is worse than reporting an unmatched title.

10. AI Generation Contract

The AI stage receives the exact WordPress title, approved industry, and resolved location context. Its output contract requests a page title, focus keyword, meta description, and a bounded keyword set in a predictable labeled format.

Generation constraints:

- Every output must remain inside the selected business industry.
- The title must be concise, service-specific, and location-aware when location is approved.
- The meta description must stay within the configured length boundary and avoid generic filler.
- The focus keyword must represent the core service rather than a sentence or question.
- The keyword list must meet the configured count and remain relevant to the exact page.
- Output must use plain text and avoid HTML entities or banned punctuation patterns.

Prompt instructions alone are not treated as sufficient validation. The worker parses the response and applies deterministic repair before any save operation.

11. Deterministic Output Repair

The repair layer normalizes the focus keyword, removes banned or malformed copy, enforces title length, bounds meta-description length, checks industry relevance, and supplies controlled fallbacks when required fields are missing or unusable.

A fallback is not a license to invent unsupported business claims. It uses conservative templates and approved context to produce syntactically valid metadata while preserving the exact service and location boundaries supplied to the job.

Before save, verify:

- The resolved WordPress item still matches the intended target.
- The focus keyword is non-empty and service-specific.
- The title and description meet configured length rules.
- No wrong-industry keywords, raw labels, markup, or placeholder tokens remain.
- The keyword list is parseable and bounded.

12. AIOSEO Metadata Persistence

The persistence stage reads the existing AIOSEO record when needed, constructs the supported metadata payload, and writes the SEO title, description, focus keyphrase, keyword collection, and aligned social metadata. It also builds keyphrase analysis data used by the plugin's editing experience.

The AIOSEO update is the primary success criterion for an item. Optional tag assignment is reported separately so a page can show a successful metadata update even when the WordPress user lacks tag permission or a tag extension is unavailable.

A save response should be validated by status and, for controlled samples, by reading the item back through an independent WordPress or editor view. A successful HTTP request alone does not prove that every cache and rendered page has refreshed.

13. WordPress Tags and Page Support

Posts support tags through standard WordPress behavior. Pages do not always expose the same taxonomy. For page runs, the worker checks whether tags are enabled and can attempt approved setup when the site and user permit it.

Tag names are derived from the validated keyword list. Existing tags are reused when found; missing tags can be created and their identifiers assigned to the target item. Concurrent creation improves speed, but duplicate-name races and site permissions must be handled explicitly.

Tag outcome categories:

- Assigned - the tag identifiers were saved to the item.
- Reused - existing site tags were matched rather than duplicated.
- Skipped - metadata succeeded but tagging was unsupported or unauthorized.
- Failed - tag creation or assignment returned an error requiring review.

14. Page, Post, and Content-Creation Modes

Page and post optimization share the same discovery, generation, validation, and AIOSEO core while using the appropriate WordPress endpoint. Draft and published items can be included so approved metadata may be prepared before publication.

The automation module also contains an optional content-creation capability for creating a draft item when explicitly requested. That is a materially different action from optimizing existing content and must remain separately authorized, visible, and tested. An unmatched title must not silently create a new page.

14A. Optional Planning and Article Generation (added July 2026)

Beginning in July 2026 the tool gained an optional planning-and-authoring capability layered around the stable metadata workflow described above. The legacy Pages and Posts metadata

path is unchanged: an absent or default choice runs exactly the request it always did. Every addition below is opt-in and cannot alter the protected path.

14A.1 Independent Heatmap Analyzer

An optional analyzer accepts a local-ranking heatmap export and returns selectable keyword and article-topic suggestions with supporting evidence, an explicit confidence label, and no ranking-position guarantees. Analysis is independent of article creation; uploading and analyzing a file never mutates WordPress or changes a later metadata run. A free, parse-only pre-inspection reads the file and returns its own service list plus data-health diagnostics (grid offset, weak-point count, and rank spread) with zero AI or third-party calls, so obvious defects in an export surface before any paid processing.

- Deterministic multi-section parsing with recovery for export-specific quoting quirks.
- Selectable suggestions can be copied into the existing Keywords field; the user chooses one to five.
- Every suggestion carries evidence and an honest confidence label; measured language is reserved for the one measured query present in the export.

14A.2 Geo Relevance Planner

The planner uses the latitude and longitude already present in a heatmap export. It selects grid points at or worse than a configurable rank threshold, clusters them, and computes true distance and compass bearing from the business location. Each cluster is resolved to real, verified landmarks through a mapping provider (Google Places API, New). Landmark names are never invented by the language model: the model receives the verified landmark list as data and must copy names exactly, or fall back to honest address-derived rows when none are available. The provider is bounded by a per-job call cap, a coordinate cache, and a short timeout, and degrades safely to the previous behavior when it is disabled or unavailable, so a lookup failure can never fail an analysis.

14A.3 Full-Article Generation for Pages and Posts

When enabled, the tool can create up to five complete WordPress article drafts from selected keywords, for either Pages or Posts. The choice defaults to No, and for either content type No submits the unchanged legacy request. Yes reveals per-article configuration. Content type is validated against a fixed allow-list and mapped to the correct REST collection, so a supplied value can never be concatenated into an endpoint.

- One selected keyword produces one article; the batch minimum is one and the maximum is five.
- Primary-keyword mentions and link counts are coupled and configurable per article, within validated ranges.
- Every link uses a user-supplied exact URL; internal and external link counts are enforced independently, and external links must be distinct.
- New drafts default to draft status until publishing is explicitly approved.

- Optional global and per-article instructions are carried as untrusted data and cannot override protected rules.

14A.4 Article Content and Factual Controls

Generated drafts are validated deterministically before any WordPress write. Invalid output is repaired within a retry limit or rejected, and success is never reported for an article that fails validation. Enforced rules include a minimum length, exactly seven FAQs, the exact business name, phone, and full address on every mention, sanitized WordPress-safe HTML, a maintained banned-word list, and a prohibition on invented history, testimonials, medical claims, local facts, or citations. External links open in a new tab with safe rel attributes; internal links stay in the same tab.

14A.5 July 2026 Interface and Article Refinements

A set of client-approved refinements followed initial delivery. Some deliberately reverse earlier rules at the client's instruction and are recorded here so the current behavior is unambiguous.

- Action buttons changed from purple to green; the brand mark is unchanged.
- Run results appear in a closable popup instead of a fixed side panel, freeing the working area.
- Keyword tags derive a city-level location (City ST) from the business address; the full address is still used verbatim in article bodies.
- Internal links within a single article may repeat the same URL when chosen; external links remain distinct.
- The primary keyword appears once as plain, unlinked text in the first paragraph, with the remaining mentions as plain links.
- The complete business address is required on every mention, not only once.

Every change above was specified before implementation, verified through the real deployed flow, and confirmed against stored WordPress content on a non-client sample site. The protected Pages, Posts-No, and legacy metadata paths remain unchanged.

14A.6 Plan by Address Without a Heatmap (added July 2026)

An optional path that produces the same neighborhood planning result as the Independent Heatmap Analyzer (14A.1) and Geo Relevance Planner (14A.2), but without requiring a heatmap CSV upload. It exists because third-party heatmap exports are sometimes unreliable: one supplied export was centered far from the business and contained no weak-coverage points, leaving the planner nothing usable to work with.

Three optional inputs sit directly above the CSV upload area - Keywords (the main service), Full Address, and Radius (in miles or kilometers). Both paths remain available: the CSV analyzer when a good export exists, and the three boxes when it does not. The existing CSV workflow is unchanged.

How it works: the business address is converted to coordinates through Google Places Text Search, reusing the existing Places integration so no additional Google service must be enabled. A single nearby-landmarks lookup is performed within the chosen radius, and the

verified landmarks are passed to the AI as data. The AI returns the same selectable suggestion rows as the CSV path - target keyword, service, location, rating, and evidence - which can be added to the Keywords field in one action.

Honest boundary: this path has no measured ranking data, because there is no grid of scanned points. Every row is labeled either places-verified (a real, Google-confirmed landmark) or AI-inferred (derived from the address), and never measured. Each row states plainly that no ranking was measured. This reflects a deliberate client decision to prioritize accurate neighborhood coverage over per-keyword rank precision, without adding a paid ranking-data provider. Landmark names always come from Google and are never invented by the AI.

Isolation and safety: the feature is a separate request path from the heatmap analyzer, so page and post optimization, article generation, and CSV analysis are all unchanged. Geocoding and landmark lookups fail safe - if an address cannot be resolved or the provider is unavailable, the request returns a clear message rather than an error, and a provider problem can never interrupt other work.

Verified live: for a real business address, the planner returned twenty suggestions built on genuine nearby landmarks - for example an urgent-care clinic about a third of a mile west, a supermarket under two miles north, and a shopping mall roughly three miles away - all within the selected radius and none claiming a measured rank. This is the intended correction to the earlier unreliable export: the suggested neighborhoods are anchored to the real business location instead of a mis-scanned area.

15. Batch Processing and Concurrency

A job can process multiple titles, with each item generating its own result row. AI generation and WordPress writes remain bounded so a large batch does not overwhelm the site or provider. Tag lookup and creation can use controlled concurrency because individual tag requests are independent.

Concurrency safeguards:

- Bound batch size and keyword count at the input boundary.
- Apply network timeouts to WordPress, AI, and browser-assisted work.
- Retry only operations that are safe to repeat.
- Use stable names and post-save reads to reduce duplicate tag races.
- Report partial completion rather than converting one item failure into a false all-or-nothing result.

16. Job Status, Results, and Reporting

The API records job identifier, lifecycle state, start and finish time, content type, target count, and safe log location. The status response exposes progress and parses terminal output into summary counts and per-item results.

CSV reports retain page identifiers, names, generated title and description lengths, keyword counts, status, and warning information. Client-facing exports should minimize personal or access data and must never include secrets, raw authentication headers, or full command lines.

Result categories:

- Updated - validated SEO metadata was saved successfully.
- Skipped - the item already met the configured optimization rule or optional work was not required.
- Not found - the submitted title did not resolve to a safe exact target.
- Failed - the item could not complete generation, validation, permission, or persistence.
- Timeout - the job exceeded its execution boundary and requires reconciliation before retry.

17. Frontend, API, and Health Separation

The PHP page can load while the Python worker is unavailable. The interface therefore performs an authenticated health check and reports backend reachability separately from frontend availability.

The watchdog follows the same principle. It tests the health endpoint first, checks whether a matching process already exists, and starts the service only when health is down and the process is absent. This avoids the earlier failure mode in which a stale or misleading process-name check reported health without proving that the API could answer requests. Health should remain lightweight and must not expose configuration values. A stronger readiness check may verify required source availability and dependency initialization without making live WordPress or AI calls on every poll.

18. Failure Semantics and Recovery

- Frontend available, API unavailable - block new jobs, show API offline, and alert operations.
- WordPress preflight fails - stop before generation or writes and report access or compatibility failure.
- AIOSEO API unavailable - do not claim optimization success; report the missing dependency.
- Title not found - report the exact unmatched input and do not choose a different item.
- AI output invalid - repair deterministically or fail the item; never save raw malformed output.
- Tag permission missing - keep successful AIOSEO metadata distinct from best-effort tag status.
- Job timeout - reconcile saved item state before retrying to avoid duplicate or conflicting operations.
- One target fails - continue independent targets when safe and return a partial result.

19. Security and Privacy Boundaries

The tool handles privileged WordPress access and therefore requires stronger controls than an ordinary public form. Credentials belong in a protected job boundary and approved secret

mechanisms, not in documentation, source-control history, URLs, browser logs, screenshots, CSV files, or long-lived diagnostic output.

Required controls:

- Protect the operator interface with strong authentication, rate limiting, and session controls.
- Require HTTPS at the public boundary and restrict the worker API to local or private access.
- Use least-privilege WordPress application credentials and rotate them regularly.
- Redact secrets from exceptions, subprocess arguments, process inspection, job logs, and returned status data.
- Validate the submitted site destination to reduce server-side request forgery and unintended internal access.
- Restrict plugin installation and interactive administrator actions to separately approved operations.
- Delete temporary input and job artifacts according to a defined retention policy.
- Audit who started a job, which site and items were in scope, and what changed without recording secret values.

20. Observability and Operational Health

Monitoring should distinguish the public interface, backend API, WordPress dependency, AI dependency, and item-level save outcomes. Process uptime alone is not enough; the useful measure is whether authorized jobs can reach terminal states with correct results.

Recommended signals:

- Frontend health and authenticated backend health.
- Job queue age, running duration, terminal state, and timeout rate.
- WordPress preflight failures by safe site label.
- Content match rate and unmatched-title patterns.
- AI parse and repair frequency.
- AIOSEO save success, read-back verification, and tag outcome.
- Local-to-live source parity and watchdog behavior after deployment.
- Credential-redaction checks for logs and client-facing exports.

21. Test Strategy

21.1 Input and matching

Test empty and malformed input, page versus post selection, one and many titles, HTML entities, smart punctuation, dash variations, repeated whitespace, exact not-found behavior, and duplicate titles.

21.2 Generation and repair

Use fixed AI responses to test missing labels, overlong titles, short or overlong descriptions, banned punctuation, wrong-industry keywords, missing location, excessive keywords, and deterministic fallback behavior.

21.3 WordPress and AIOSEO

Cover preflight failure, permission denial, AIOSEO absence, page and post updates, existing and new tags, tag permission failure, drafts, published items, already-optimized items, and post-save read-back.

21.4 Jobs and operations

Cover API offline, background start failure, in-progress polling, completion, partial failure, timeout, lost in-memory job state, watchdog restart, source parity, and safe retry after uncertain completion.

21.5 Security checks

Verify unauthorized interface requests fail, worker access remains private, destination validation rejects unsafe targets, and no credentials appear in logs, reports, error messages, process records, or exported documentation.

22. Deployment and Version Integrity

1. Capture the current live PHP, API, worker, dependency, and watchdog sources before editing.
2. Apply the smallest scoped change to the local source of truth.
3. Run syntax, deterministic, and integration tests appropriate to the changed component.
4. Deploy through the approved server process without copying secret files into artifacts.
5. Compare local and live source hashes or equivalent immutable evidence.
6. Restart or reload the affected service and verify both local and authenticated public health.
7. Run an unchanged-path smoke check plus one controlled SEO job when risk permits.
8. Retain a rollback artifact and record the verification result.

23. Incident Lessons Incorporated into the Design

23.1 The page was online while the API was offline

A successful PHP response masked a failed Python backend. Frontend availability and worker health are now separate checks, and the interface communicates API reachability directly.

23.2 Process-name monitoring gave weak assurance

A watchdog that only searches for a process can miss an unhealthy or incorrectly named service. The current approach tests the health endpoint first and starts only when health and process evidence justify it.

23.3 Title normalization was asymmetric

Normalizing only WordPress titles left equivalent operator input unmatched. Both sides now share one normalization contract, including entity and punctuation handling.

23.4 Tag creation exceeded practical timing

Sequential tag work caused avoidable delay. Independent tag requests were made concurrent, while AIOSEO metadata success remains separate from optional tag completion.

23.5 AI examples leaked the wrong business context

Examples and prompts can pull generation toward an unrelated industry. Industry locking, exact title context, location context, and deterministic repair now constrain every output.

24. Operational Playbook

Before running a client site:

- Confirm authorization, site ownership, content scope, industry, and approved location context.
- Create a least-privilege WordPress application credential and verify required plugins.
- Confirm the backend health check passes and no older job for the same items is still running.
- Start with one controlled page or post and review saved results.

After a job:

- Review updated, skipped, unmatched, failed, and timeout counts.
- Spot-check WordPress and AIOSEO fields rather than relying only on the generated report.
- Review wording for branding, claims, location, grammar, and duplication.
- Reconcile uncertain items before retrying and revoke temporary access when no longer needed.

After any deployment:

- Verify source parity, API health, watchdog behavior, and one unchanged path.
- Scan logs and exports for secret leakage.
- Record the change, verification evidence, and rollback point.

25. Known Limitations and Risk Register

- Search outcomes - valid metadata does not guarantee rankings, indexing, traffic, or conversions.
- External dependencies - WordPress, AIOSEO, AI service, DNS, network, and hosting failures can block or delay jobs.
- In-memory job state - an API restart can lose status for active or recently completed jobs even if worker artifacts remain.
- Credential exposure surface - process launch, diagnostics, and temporary files require further hardening and routine redaction verification.
- Tag races - concurrent create operations can encounter duplicate names or permission differences.
- Generated claims - AI output still requires client review, especially for regulated services, outcomes, credentials, pricing, or location statements.
- Plugin compatibility - WordPress and AIOSEO versions can change API behavior and field expectations.
- Interactive score refresh - browser-assisted editor actions require stronger access and are more fragile than REST metadata saves.
- Historical workflow divergence - n8n sources and current Python behavior can drift unless their ownership is made explicit.

26. Recommended Next Engineering Gates

1. Move durable job state to a small persistent store with expiry and restart recovery.
2. Remove secrets from process command lines and enforce structured redaction in every log and status path.
3. Add strict destination allowlisting or tenant registration for WordPress sites.
4. Add post-save read-back verification and a clear metadata-versus-tag result model.
5. Add a preview and approval mode before saving AI-generated metadata to production content.
6. Add bounded worker concurrency, cancellation, retry policy, and duplicate-job protection.
7. Define one authoritative production implementation and archive or test historical workflow paths accordingly.
8. Add scheduled synthetic health and sample-job checks without using a real client credential.

27. Technology Stack and Responsibilities

27.1 Interface and API

- PHP and browser JavaScript - protected operator experience, validation, progress display, and same-origin proxying.
- Python Flask - job creation, background execution, health, status, timeout, and safe result parsing.

27.2 SEO processing

- Python automation module - preflight, matching, generation repair, WordPress and AIOSEO writes, tagging, and reporting.
- Claude - structured SEO draft generation within supplied industry and location constraints.
- WordPress REST API and AIOSEO API - content discovery and metadata persistence.

27.3 Operations and history

- Health-based watchdog - restarts the API only when health and process evidence require it.
- CSV and job logs - operational evidence subject to retention and redaction policy.
- n8n workflow sources - historical automation and regression context requiring explicit lifecycle ownership.

28. Current Status Summary

FTM SEO Automation is a live-maintenance tool for applying structured SEO updates across authorized WordPress sites. Its current runtime separates a protected PHP interface from a supervised Python job API and a WordPress automation worker.

The strongest implementation qualities are exact target matching, industry-locked generation, deterministic output repair, AIOSEO-specific persistence, page and post support, asynchronous job status, result reporting, and health-based backend supervision.

The system should be presented as an automation accelerator with operational controls, not as an autonomous ranking guarantee. Safe value depends on least-privilege access, client approval, exact content scope, dependency health, secret handling, monitoring, and verification after each change.

Appendix A - Representative Request Traces

A.1 Valid page optimization

```
authorized job -> WordPress preflight -> exact page match -> SEO generation -> repair ->
AIOSEO save -> report updated
```

A.2 Equivalent title punctuation

```
submitted title -> shared normalization -> stored title match -> correct page ID -> normal
processing
```

A.3 Unmatched title

```
submitted title -> no exact normalized match -> not-found result -> no guessed page -> no
write
```

A.4 Metadata succeeds, tags unavailable

AIOSEO save succeeds -> tag permission fails -> metadata updated -> tag status skipped or failed -> partial result shown

A.5 Backend offline

interface health poll -> API unreachable -> new run blocked -> watchdog and operator alert -> no false processing state

A.6 Job timeout

job exceeds limit -> timeout state -> saved-item reconciliation -> safe retry decision

Appendix B - Safe Site-Onboarding Checklist

1. Confirm written authorization and approved content scope.
2. Verify WordPress and AIOSEO versions and required API availability.
3. Create a dedicated least-privilege application credential.
4. Register or allowlist the site through the approved operational process.
5. Confirm industry, location, title format, keyword count, and wording constraints.
6. Run health and preflight without saving.
7. Optimize one controlled draft or approved sample item.
8. Review WordPress, AIOSEO, tags, social metadata, and the report.
9. Approve the batch size and monitor the first production run.
10. Rotate or remove temporary access after completion.

Appendix C - Glossary

- AIOSEO - the WordPress SEO plugin and API used to store metadata and analysis fields.
- Application credential - a revocable WordPress credential intended for API access rather than interactive reuse.
- Exact normalized match - equality after both titles pass through the same entity, punctuation, whitespace, and case rules.
- Focus keyphrase - the primary service phrase the page is optimized around.
- Deterministic repair - code-enforced correction applied after AI generation and before persistence.
- Preflight - read-only checks that confirm connectivity, permissions, content type, and required API support.
- Terminal state - completed, failed, error, or timeout; running is not terminal.
- Read-back verification - retrieving the saved record independently to confirm persistence.
- Public technical edition - documentation designed for professional review without private access material.

Appendix D - Review Checklist

For client review:

- Confirm industry, location, service wording, claims, brand tone, and target content.
- Confirm the tool's ranking limitations and the need for editorial review.
- Confirm authorization, credential ownership, access removal, and report retention.

For engineering review:

- Trace one target through input validation, exact matching, generation, repair, save, tags, and reporting.
- Verify page and post behavior, failure isolation, timeout reconciliation, and post-save read-back.
- Verify the current runtime and historical n8n workflow are not conflated.

For security and operations review:

- Confirm the worker remains private and submitted destinations are constrained.
- Confirm no secrets appear in process arguments, logs, reports, errors, screenshots, or exports.
- Confirm health monitoring, source parity, rollback evidence, and job-retention policy.

Developed by Zahidul Islam

SEO and backend automation - WordPress integration, AI output controls, operational reliability, and client-safe delivery.